

Lukasz

Osoba 1 - Matematyka 3D, graf sceny, budowa pokoju

Materiały do obrony projektu: Silnik 3D + Strzelnica (FreeGLUT / OpenGL)

Twoje pliki: Engine/Math3D.h, Engine/Math3D.cpp, Engine/SceneNode.h, Engine/SceneNode.cpp, Demo/Room.cpp

0. Gdzie sa moje pliki (mapa do prezentacji)

Ponizsze drzewo pokazuje caly projekt. Strzałka <<< wskazuje pliki, ktore robilem ja (Lukasz). W trakcie pokazu otwieraj wlasnie te.

```
pgk/
|
+-- Engine/                                <- silnik (rdzen)
|   +-- Math3D.h                          <<< MOJE  (wektory, macierze, raycasty)
|   +-- Math3D.cpp                        <<< MOJE
|   +-- SceneNode.h                      <<< MOJE  (graf sceny - drzewo obiektow)
|   +-- SceneNode.cpp                    <<< MOJE
|   +-- Camera.* Light.* Texture.*       (Kacper)
|   +-- PrimitiveNode.*                  (Rogert)
|   +-- Engine.*                         (Jakub)
|
+-- Demo/                                  <- gra
|   +-- Room.cpp                          <<< MOJE  (sciany + dekoracje pokoju)
|   +-- Obstacles.cpp                     (Rogert)
|   +-- Targets.cpp                       (Kacper)
|   +-- Gameplay.cpp Constants.h ShootingGallery.h (Jakub)
|
+-- main.cpp CMakeLists.txt README.md    (Jakub)
```

Najwazniejsze do zapamietania: ja odpowiadam za **matematyke** (wszystko co liczy wektory i macierze), za **strukture sceny** (jak obiekty sa ulozone w drzewo) i za **zbudowanie pomieszczenia** w grze.

1. O co chodzi w moich plikach (po ludzku)

Silnik 3D musi cos policzyc, zanim cokolwiek narysuje. Zeby pokazac szescian w odpowiednim miejscu i pod odpowiednim katem, trzeba mnozyc **macierze** i przekształcac **wektory**. To wlasnie robi **Math3D** - to nasz wlasny, reczny zestaw narzedzi matematycznych (nie uzywamy gotowej biblioteki jak glm).

SceneNode to z kolei sposob, w jaki trzymamy obiekty sceny. Zamiast luznej listy, obiekty tworza **drzewo** (graf sceny). Dzieki temu jak ruszymy rodzicem, dzieci ruszaja sie razem z nim.

Room.cpp to juz czesc gry - uzywam swoich klas (i prymitywow kolegow), zeby zbudowac pokoj: podloge, sufit, 4 sciany i dekoracje.

2. Math3D.h / Math3D.cpp - matematyka silnika

2.1. Vec3 - wektor 3D

Wektor to po prostu trzy liczby: x, y, z. Moze oznaczac **punkt** w przestrzeni albo **kierunek**. Zdefiniowalem podstawowe operacje (dodawanie, odejmowanie, mnozenie przez liczbe) plus funkcje matematyczne.

Math3D.cpp - kluczowe operacje na wektorach

```
float dot(const Vec3& a, const Vec3& b) {
    return a.x*b.x + a.y*b.y + a.z*b.z;    // iloczyn skalarny
}

Vec3 cross(const Vec3& a, const Vec3& b) { // iloczyn wektorowy
    return Vec3(a.y*b.z - a.z*b.y,
                a.z*b.x - a.x*b.z,
                a.x*b.y - a.y*b.x);
}

float length(const Vec3& v) { return std::sqrt(dot(v, v)); }

Vec3 normalize(const Vec3& v) {
    float len = length(v);
    if (len <= 0.000001f) return Vec3(0,0,0); // ochrona przed dzieleniem przez 0
    return v / len;
}
```

dot (iloczyn skalarny) mowi nam m.in. o kacie miedzy wektorami. **cross** (iloczyn wektorowy) daje wektor **prostopadly** do obu - uzywam go np. do policzenia osi kamery. **normalize** skraca wektor do dlugosci 1, zachowujac kierunek (wektory kierunku prawie zawsze chcemy znormalizowane).

2.2. Mat4 - macierz 4x4 (najwazniejsza rzecz!)

Macierz 4x4 to 16 liczb, ktore opisuja przekształcenie: przesuniecie, obrot, skalowanie albo rzutowanie. Trzymam je w tablicy **column-major** (kolumnami), bo tego wymaga OpenGL.

Math3D.cpp - dostep do elementu macierzy

```
float& Mat4::at(int row, int col) { return m[col * 4 + row]; }
//                                     ^^^^^^^^^^^^^^^^^
//   element (row,col) jest pod indeksem col*4+row => column-major
```

Dlaczego column-major? Bo funkcja OpenGL *glLoadMatrixf* czyta dane wlasnie w tej kolejnosci. Gdybym trzymal je wierszami (row-major), obraz bylby kompletnie przekrecony. To wazne - wykladowca moze o to spytac.

Macierz lookAt (i slynny bug, ktory naprawilismy)

lookAt tworzy macierz widoku - ustawia 'oko' kamery w punkcie *eye*, kierując je na *center*. Liczymy trzy osie kamery: *forward* (do przodu), *side* (w prawo) i *trueUp* (w gore).

Math3D.cpp - lookAt (wersja poprawna)

```
Vec3 forward = normalize(center - eye);
Vec3 side    = normalize(cross(forward, up));
Vec3 trueUp  = cross(side, forward);

Mat4 result = Mat4::identity();
result.at(0,0)=side.x; result.at(0,1)=side.y; result.at(0,2)=side.z; // wiersz 0
result.at(1,0)=trueUp.x;result.at(1,1)=trueUp.y;result.at(1,2)=trueUp.z; // wiersz 1
result.at(2,0)=-forward.x; result.at(2,1)=-forward.y; result.at(2,2)=-forward.z;
result.at(0,3)=-dot(side,eye);
result.at(1,3)=-dot(trueUp,eye);
result.at(2,3)= dot(forward,eye);
```

Bug, który mieliśmy: wektory osi były wpisane jako **kolumny** zamiast **wierszy**. Dla obrotu transpozycja = odwrotność, więc efekt był taki, że kamera 'krazyła wokół jakiegoś dziwnego punktu', a strzały nie trafiały w cele (bo kierunek patrzenia nie zgadzał się z tym, co widac). Przy yaw=pitch=0 bug był niewidoczny, bo zamieniane pola były zerami - dlatego długo go nie zauważyliśmy.

2.3. Funkcje przecięć promienia (raycasty)

To one sprawiają, że w grze można w coś 'trafic'. Promień to po prostu: **$P(t) = \text{origin} + t * \text{kierunek}$** . Szukamy najmniejszego dodatniego *t*, przy którym promień dotyka obiektu.

Math3D.cpp - przecięcie promienia ze sferą

```
bool raySphereIntersect(const Vec3& origin, const Vec3& dir,
                        const Vec3& center, float radius, float& outT) {
    Vec3 oc = origin - center;
    float a = dot(dir, dir);
    float b = dot(oc, dir);
    float c = dot(oc, oc) - radius*radius;
    float disc = b*b - a*c;           // wyznik równania kwadratowego
    if (disc < 0.0f) return false;    // brak przecięcia
    float sqrtD = std::sqrt(disc);
    float t1 = (-b - sqrtD) / a;      // bliższe
    float t2 = (-b + sqrtD) / a;      // dalsze
    if (t1 > 0.0001f) { outT = t1; return true; }
    if (t2 > 0.0001f) { outT = t2; return true; }
    return false;
}
```

Do zwykle **równanie kwadratowe**: podstawiamy $P(t)$ do równania sfery $|P - \text{center}|^2 = r^2$ i wychodzi $a*t^2 + 2*b*t + c = 0$. Jeżeli wyznik (*disc*) jest ujemny - promień mija sferę. Mam też analogiczne funkcje dla walca (*rayCylinderIntersect*), stożka (*rayConeIntersect*) i prostopadłościanu (*rayAABBIntersect* - metoda 'slab'). Kacper i Rogert używają ich w grze.

3. SceneNode.h / SceneNode.cpp - graf sceny

Graf sceny to **drzewo** obiektów. Każdy obiekt (SceneNode) ma rodzica i listę dzieci. Dzięki temu transformacje się **dziedziczą**: jak obroci rodzica, to dzieci obraca się razem z nim. To standard w silnikach 3D.

3.1. Macierz lokalna

SceneNode.cpp - sklejenie pozycji, obrotu i skali w jedną macierz

```
Mat4 SceneNode::localMatrix() const {
    return Mat4::translation(localPosition)
        * Mat4::rotationY(localRotation.y)
        * Mat4::rotationX(localRotation.x)
        * Mat4::rotationZ(localRotation.z)
        * Mat4::scale(localScale);
}
```

Kolejność mnożenia ma znaczenie! Czytając od prawej: najpierw **skalujemy** obiekt, potem **obracamy** (Z, X, Y), na końcu **przesuwamy**. Gdyby przesunięcie było pierwsze, obiekt obracałby się wokół środka świata, a nie wokół siebie.

3.2. Rekurencyjne rysowanie

SceneNode.cpp - przejście po całym drzewie sceny

```
void SceneNode::renderRecursive(const Mat4& parentMatrix) const {
    Mat4 world = parentMatrix * localMatrix(); // moja macierz świata
    if (nodeVisible) renderSelf(world);        // narysuj siebie (jeżeli widoczny)
    for (const auto& child : childNodes)       // potem wszystkie dzieci
        child->renderRecursive(world);
}
```

Silnik wola te funkcje na korzeniu (root) z macierzą jednostkową. Funkcja schodzi w dół drzewa, mnożąc macierze. **renderSelf** jest puste w klasie bazowej - dopiero klasy potomne (szescian, sfera, światło) nadpisują je i rysują geometrie. To przykład **polimorfizmu** i wzorca, gdzie klasa bazowa zarządza strukturą, a potomne definiują szczegóły.

nodeVisible - jak ustawimy na false, obiekt się nie rysuje (ale jego dzieci tak). Kacper używa tego do 'puli celów' - chowamy sfery zamiast je usuwać.

4. Demo/Room.cpp - budowa pomieszczenia

Tu używam wszystkiego powyżej plus prymitywów Rogerta (PlaneNode, SphereNode...). Buduje pokój o wymiarach 20 x 6 x 30 metrów: podłogę, sufit i 4 ściany, każda jako **PlaneNode** obrocony w odpowiednią stronę.

Room.cpp - przykład jednej ściany (sufit)

```
// sufit: płaszczyzna obrócona o 180 stopni (PI) wokół X, żeby normalna
// patrzyła w dół (do wnętrza pokoju)
ceiling_>setPosition(Vec3(0, ROOM_HEIGHT, ROOM_Z_CENTER));
ceiling_>setRotation(Vec3(PI, 0, 0));
ceiling_>setUVScale(FLOOR_UV_SCALE);
ceiling_>setMaterial(ceilingMat);
ceiling_>setTexture(ceilingTex_);
```

buildDecorations() dorzuca dekoracje: kule i kostki na ścianach, torus (zyrandol) pod sufitem. Używam lambda (makeSphere, makeCube, makeCylinder), żeby nie powtarzać kodu. Dekoracje są wysoko (y >= 3), więc gracz pod nimi przechodzi i nie ma z nimi kolizji.

Tekstury sa albo wczytane z pliku BMP (jezeli istnieje), albo wygenerowane proceduralnie - np. *generatePerlinNoise* dla sufitu. Te funkcje napisal Kacper, ja ich tylko uzywam.

5. Pytania od wykładowcy - przygotowanie

Przy każdym pytaniu jest pasek **Gdzie** - mówi w którym pliku i w której funkcji szukać odpowiedzi, żebyś w trakcie obrony szybko otworzył właściwe miejsce.

Math3D - wektory (Vec3)

P: Co to jest iloczyn skalarny (dot) i co nam mówi?

O: $\text{dot}(a,b) = a_x \cdot b_x + a_y \cdot b_y + a_z \cdot b_z$. Jest równy $|a| \cdot |b| \cdot \cos(\text{kat})$. Gdy wektory są znormalizowane, dot to wprost cosinus kąta między nimi - mówi czy wskazują w tę samą stronę (dodatni), prostopadle (0) czy przeciwnie (ujemny). Używam go też do liczenia długości: $\text{length} = \sqrt{\text{dot}(v,v)}$.

Gdzie: Math3D.cpp -> dot(), length()

P: Co robi cross (iloczyn wektorowy) i gdzie go używasz?

O: cross daje wektor prostopadły do dwóch podanych. Używam go w lookAt: mając forward i przybliżoną górę up, liczę prawo kamery $\text{side} = \text{cross}(\text{forward}, \text{up})$, a potem dokładną górę $\text{trueUp} = \text{cross}(\text{side}, \text{forward})$. Tak buduje 3 prostopadłe osie kamery.

Gdzie: Math3D.cpp -> cross() oraz Mat4::lookAt()

P: Dlaczego w normalizacji sprawdzasz długość przed dzieleniem?

O: Żeby nie dzielić przez zero. Wektor zerowy ma długość 0 - dzielenie dałoby NaN/nieskończoność. Sprawdzam `if (len <= 0.000001f)` i zwracam wtedy wektor zerowy jako bezpieczną wartość.

Gdzie: Math3D.cpp -> normalize()

P: Czemu length używa $\sqrt{\text{dot}(v,v)}$ zamiast osobnego wzoru?

O: $\text{dot}(v,v) = x^2 + y^2 + z^2$, czyli dokładnie to, co jest pod pierwiastkiem we wzorze na długość. Więc $\sqrt{\text{dot}(v,v)} = \sqrt{x^2 + y^2 + z^2}$. Użycie dot to po prostu mniej powtórzonego kodu.

Gdzie: Math3D.cpp -> length()

Math3D - macierze (Mat4)

P: Dlaczego trzymasz macierz jako tablicę 16 floatów, a nie 4x4?

O: Bo OpenGL (`glLoadMatrixf` / `glMultMatrixf`) przyjmuje płaską tablicę 16 floatów. Trzymanie ich od razu w takiej formie pozwala podać wskaźnik `data()` wprost do OpenGL bez żadnej konwersji.

Gdzie: Math3D.h -> `struct Mat4 {float m[16];}`; Math3D.cpp -> `data()`

P: Co dokładnie robi funkcja `at(row, col)`? Wytłumacz wzór `col*4+row`.

O: `at` zwraca element macierzy w danym wierszu i kolumnie. Wzór `m[col*4+row]` oznacza, że przechodząc przez pamięć po kolei dostajemy: całą kolumnę 0 (4 liczby), potem kolumnę 1 itd. To właściwie definicja column-major. Gdyby było `row*4+col`, byłoby row-major.

Gdzie: Math3D.cpp -> `Mat4::at()` (linia `m[col*4+row]`)

P: Co to znaczy column-major i dlaczego to ważne?

O: Column-major znaczy, że kolejne 4 liczby w pamięci to jedna kolumna macierzy. OpenGL tego wymaga. Gdyby trzymać dane wierszami (row-major), macierz byłaby transponowana i wszystkie transformacje byłyby złe - obraz kompletnie przekrecony.

Gdzie: Math3D.cpp -> Mat4::at()

P: Jak zbudowana jest macierz translacji? Czemu offset jest w 4. kolumnie?

O: Translacja to jednostkowa macierz z wpisanym przesunięciem w kolumnie 3 ($at(0,3)=x$, $at(1,3)=y$, $at(2,3)=z$). Przy mnożeniu macierz*punkt te wartości dodają się do współrzędnych, bo punkt ma czwartą współrzędną $w=1$. Dlatego translacja siedzi właśnie w ostatniej kolumnie.

Gdzie: Math3D.cpp -> Mat4::translation()

P: Jak działa macierz rotacji wokół Y? Skąd cos i sin w konkretnych miejscach?

O: rotationY wstawia cos i sin tak, by obrócić współrzędne X i Z (Y zostaje). $at(0,0)=c$, $at(0,2)=s$, $at(2,0)=-s$, $at(2,2)=c$. To standardowa macierz obrotu - punkt (x,z) przechodzi na $(x*\cos+z*\sin, -x*\sin+z*\cos)$. Znaki decydują o kierunku obrotu.

Gdzie: Math3D.cpp -> Mat4::rotationY() (analogicznie rotationX/Z)

P: Jak działa mnożenie macierzy w waszym kodzie?

O: Potrójna pętla: dla każdego wiersza i kolumny wyniku sumuje iloczyny $left.at(row,i)*right.at(i,col)$ po i od 0 do 3. To klasyczna definicja mnożenia macierzy - wiersz lewej przez kolumnę prawej.

Gdzie: Math3D.cpp -> operator*(Mat4, Mat4)

P: Czym różni się transformPoint od transformVector?

O: transformPoint traktuje argument jak punkt ($w=1$) - uwzględnia translację (4. kolumnę) i dzieli przez w na końcu (dla perspektywy). transformVector traktuje go jak kierunek ($w=0$) - pomija translację, bo kierunku się nie przesuwa, tylko obraca/skaluje.

Gdzie: Math3D.cpp -> transformPoint(), transformVector()

P: Po co w transformPoint dzielenie przez w na końcu?

O: Przy rzutowaniu perspektywnym czwartą współrzędną w przestaje być 1. Dzielenie x/w , y/w , z/w to 'dzielenie perspektywiczne' - to ono sprawia, że obiekty dalej wyglądają mniejsze. Sprawdzam $fabs(w)>\epsilon$, żeby nie dzielić przez zero.

Gdzie: Math3D.cpp -> transformPoint()

Math3D - projekcje i lookAt

P: Co robi macierz perspective i co oznacza $f = 1/\tan(\text{fov}/2)$?

O: perspective buduje macierz rzutu perspektywicznego. $f=1/\tan(\text{fov}/2)$ to 'ogniskowa' - im wezsze pole widzenia (fov), tym wiekszy f, tym bardziej przyblizone. Dziele f przez aspect w $\text{at}(0,0)$, zeby obraz nie byl rozciagniety na szerokim oknie. $\text{at}(3,2)=-1$ przenosi -z do w (dzielenie perspektywiczne).

Gdzie: Math3D.cpp -> Mat4::perspective()

P: Po co osobna macierz orthographic skoro jest perspective?

O: Ortogonalna nie ma perspektywy - obiekty maja ten sam rozmiar niezaleznie od odleglosci. Uzywamy jej do rysowania HUD (2D na ekranie) i do trybu rzutu rownoleglego. To rozne zastosowania niz scena 3D.

Gdzie: Math3D.cpp -> Mat4::orthographic(); uzycie w Engine drawHUD()

P: Wyjasnij dokladnie, jak lookAt buduje macierz widoku.

O: Najpierw licze 3 osie kamery: $\text{forward}=\text{normalize}(\text{center}-\text{eye})$, $\text{side}=\text{normalize}(\text{cross}(\text{forward},\text{up}))$, $\text{trueUp}=\text{cross}(\text{side},\text{forward})$. Te osie wpisuje jako WIERSZE macierzy (side w wiersz 0, trueUp w 1, -forward w 2). 4. kolumna to $-\text{dot}(\text{os},\text{eye})$ - przesuniecie, ktore przenosi eye do poczatku ukladu. Efekt: macierz obraca swiat tak, jakby kamera patrzyla wzdluz -Z.

Gdzie: Math3D.cpp -> Mat4::lookAt()

P: Dlaczego osie kamery sa w wierszach, a nie w kolumnach? (slynnny bug)

O: Bo macierz widoku to ODWROTNOSC ulozenia kamery w swiecie. Dla obrotu odwrotnosc = transpozycja, czyli osie ida w wierszach. Mielismy bug, gdzie wpisalismy je w kolumny - kamera 'krazyla wokol dziwnego punktu', a strzaly mijaly cele. Przy yaw=pitch=0 bug byl niewidoczny, bo zamieniane pola byly zerami.

Gdzie: Math3D.cpp -> Mat4::lookAt() (komentarz o transpozycji w kodzie)

P: Czemu w lookAt jest -forward, a nie forward?

O: Bo w OpenGL kamera domyslnie patrzy wzdluz osi -Z. Trzecia os kamery (w glab ekranu) to wiec -forward. Dlatego w wierszu 2 macierzy wpisuje -forward.x, -forward.y, -forward.z.

Gdzie: Math3D.cpp -> Mat4::lookAt()

Math3D - raycasty (przeciecia promienia)

P: Wyjasnij raySphereIntersect linijka po linijce.

O: Podstawiam promien $P(t)=\text{origin}+t*\text{dir}$ do rownania sfery $|P-\text{center}|^2=r^2$. Po rozwinieciu wychodzi $a*t^2+2b*t+c=0$, gdzie $oc=\text{origin}-\text{center}$, $a=\text{dot}(\text{dir},\text{dir})$, $b=\text{dot}(oc,\text{dir})$, $c=\text{dot}(oc,oc)-r^2$. Licze wyznik $\text{disc}=b^2-a*c$. Jezeli <0 - promien mija sfere. Inaczej licze $t1,t2$ i wybieram najmniejszy dodatni (najblizszy punkt przed kamera).

Gdzie: Math3D.cpp -> raySphereIntersect()

P: Dlaczego sprawdzasz $t > 0.0001f$, a nie $t > 0$?

O: Mały epsilon chroni przed błędami zaokrągleń float i przed 'trafieniem' w punkt startowy promienia ($t=0$). Bez tego mogłyby pojawiać się fałszywe przecięcia tuż przy kamerze.

Gdzie: Math3D.cpp -> wszystkie ray...Intersect (warunek $t > 0.0001f$)

P: Czemu w raySphereIntersect najpierw próbujesz t_1 , potem t_2 ?

O: $t_1 = (-b - \sqrt{D})/a$ jest zawsze mniejsze (bliższe), bo odejmujemy pierwiastek. Chcemy najbliższe trafienie przed kamerą, więc najpierw sprawdzam t_1 . Jeżeli t_1 jest ujemne (sfera za nami albo jesteśmy w środku), próbuję t_2 - dalszy punkt przecięcia.

Gdzie: Math3D.cpp -> raySphereIntersect()

P: Jak rayCylinderIntersect różni się od sfery?

O: Walec jest pionowy, więc rzutuje problem na płaszczyznę XZ (ignoruje Y w równaniu koła): $(p_x - b_x)^2 + (p_z - b_z)^2 = r^2$. To też równanie kwadratowe, ale po znalezieniu t muszę jeszcze sprawdzić, czy punkt trafienia ma Y w zakresie wysokości walca ($base.y$ do $base.y + height$) - inaczej promień przechodzi obok nad/pod walcem.

Gdzie: Math3D.cpp -> rayCylinderIntersect()

P: Jak działa rayAABBIntersect (metoda slab)?

O: Dla każdej z 3 osi liczę przedział parametru t , w którym promień jest między dwiema równoległymi ścianami (slab). Część wspólna wszystkich 3 przedziałów to przecięcie z pudełkiem. Jeżeli $t_{min} > t_{max}$ po którymś kroku - promień mija pudełko. Obsługuje też przypadek promienia równoległego do osi (dir blisko 0).

Gdzie: Math3D.cpp -> rayAABBIntersect()

SceneNode - graf sceny

P: Po co graf sceny? Nie wystarczy lista obiektów?

O: Graf pozwala na hierarchie - dziecko dziedziczy transformację rodzica. Np. światło doczepione do obiektu rusza się razem z nim. Mnożenie rodzic*dziecko robi to automatycznie. Z płaską listą trzeba by ręcznie pamiętać zależności.

Gdzie: SceneNode.h/.cpp -> cała klasa; childNodes, parentNode

P: Czemu localMatrix mnoży w kolejności $T * R * S$?

O: Czytając od prawej (tak działają macierze): najpierw skala, potem obrot ($Y * X * Z$), na końcu translacja. Dzięki temu obiekt skaluje i obraca się wokół własnego środka, a dopiero potem jest przesuwany. Odwrotna kolejność dałoby obrot wokół środka świata.

Gdzie: SceneNode.cpp -> localMatrix()

P: Dlaczego obroty sa w kolejnosci $Y * X * Z$, a nie np. $X * Y * Z$?

O: To kwestia konwencji katow Eulera. My najpierw obracamy w poziomie (Y - yaw), potem w pionie (X - pitch), potem przechylenie (Z - roll). Kolejnosć wpływa na efekt przy laczeniu obrotow, ale dla naszych obiektow (zwykle obrot tylko wokol jednej osi) jest to wystarczajace i intuicyjne.

Gdzie: `SceneNode.cpp` -> `localMatrix()`

P: Dlaczego renderRecursive mnozy parentMatrix * localMatrix, a nie odwrotnie?

O: Bo macierz swiata dziecka = macierz swiata rodzica zlozona z lokalna transformacja dziecka. Kolejnosć rodzic*dziecko sprawia, ze lokalna transformacja dziala 'wewnatrz' ukladu rodzica.

Gdzie: `SceneNode.cpp` -> `renderRecursive()`

P: Czym jest renderSelf i dlaczego jest wirtualne?

O: To metoda rysujaca konkretna geometrie. W SceneNode jest pusta (wezle grupujacy). Klasy potomne (CubeNode, PointLight...) nadpisuja ja. Wirtualnosć pozwala wywolac wlasciwa wersje przez wskaznik do klasy bazowej - to polimorfizm.

Gdzie: `SceneNode.cpp` -> `renderSelf()` (pusta); nadpisana w `PrimitiveNode/Light`

P: Co daje flaga nodeVisible i gdzie jest sprawdzana?

O: Gdy false, renderSelf jest pomijane (obiekt znika), ale dzieci nadal sie rysuja. Sprawdzam to w renderRecursive: `if (nodeVisible) renderSelf(world)`. Kacper uzywa tego do puli celow - zamiast tworzy i niszczy sfery, chowamy je.

Gdzie: `SceneNode.cpp` -> `renderRecursive()` (`if nodeVisible`); `SceneNode.h` -> `setVisible()`

P: Co sie dzieje w addChild? Czemu dziecko zapamieta rodzica?

O: `addChild` ustawia `child->parentNode = this` i dodaje dziecko do listy `childNodes`. Wskaznik na rodzica pozwala dziecku policzyc `worldMatrix` idac w gore drzewa. Sprawdzam tez czy child nie jest nullem.

Gdzie: `SceneNode.cpp` -> `addChild()`

P: Jaka jest roznica miedzy localMatrix a worldMatrix?

O: `localMatrix` to transformacja obiektu wzgledem rodzica. `worldMatrix` to transformacja wzgledem calego swiata - liczona rekurencyjnie jako `parent->worldMatrix() * localMatrix()`. Dla obiektu bez rodzica obie sa rowne.

Gdzie: `SceneNode.cpp` -> `localMatrix()`, `worldMatrix()`

Room.cpp - budowa pokoju

P: Dlaczego sufit jest obrocony o π (180 stopni) wokol X?

O: `PlaneNode` ma normalna w gore (+Y). Sufit musi swiecic do wnetrza pokoju, czyli w dol. Obrot o 180 stopni wokol osi X odwraca normalna na -Y, zeby oswietlenie liczyl sie poprawnie od spodu.

Gdzie: `Room.cpp` -> `buildRoom()` (`ceiling_->setRotation(Vec3(PI,0,0))`)

P: Jak ustawiona jest sciana boczna? Czemu obrot wokół Z?

O: Płaszczyzna jest domyślnie pozioma (XZ). Żeby postawić ją pionowo jako ścianę lewą/prawą, obracam o 90 stopni wokół osi Z (`setRotation(0,0,+PI/2)`). Tylne/przednie ściany to obrot wokół X. Pozycje ustawiam na krawędzi pokoju (`+ROOM_X_HALF`).

Gdzie: `Room.cpp` -> `buildRoom()` (`leftWall_/rightWall_ setRotation` wokół Z)

P: Co to jest setUVScale i po co go ustawiasz na ścianach?

O: Skaluje współrzędne tekstury. Domyślnie 1 powtórzenie na metr - na ścianie 30m daloby 30 drobnych kafelków. Ustawiam mniejszą wartość (0.2-0.3), żeby kafelki były większe i ładniejsze. To metoda `PlaneNode` (Rogert), ja ją tylko stosuję.

Gdzie: `Room.cpp` -> `buildRoom()` (`setUVScale`); `PlaneNode.cpp` (implementacja)

P: Po co lambdy makeSphere / makeCube / makeCylinder w dekoracjach?

O: Żeby nie powtarzać tego samego kodu (stwórz węzeł, ustaw pozycję/material/teksturę, dodaj do sceny) kilkanaście razy. Lambda to mała lokalna funkcja - wywołuje ją z różnymi pozycjami. Czystszy i krótszy kod.

Gdzie: `Room.cpp` -> `buildDecorations()` (lambda `makeSphere` itd.)

P: Czemu dekoracje nie mają kolizji?

O: Są wysoko ($y \geq 3\text{m}$), a gracz ma oczy na 1.75m i hitbox tylko w poziomie. Fizycznie nie da się w nie wejść - gracz przechodzi pod nimi. Nie trzeba dla nich liczyć kolizji, co oszczędza kod i czas.

Gdzie: `Room.cpp` -> `buildDecorations()`; por. `Obstacles.cpp` (tam są kolizje)

P: Skąd biorą się tekstury w pokoju - z pliku czy generowane?

O: Próbuje najpierw wczytać plik BMP (`loadBMP`). Jeżeli pliku nie ma albo ma zły format, funkcja zwraca `false` i wtedy generuje teksturę proceduralnie (np. `generateBricks` dla podłogi, `generatePerlinNoise` dla sufitu). Te funkcje napisał Kacper, ja ich używam.

Gdzie: `Room.cpp` -> `buildRoom()` (`if(!loadBMP) generate...`); `Texture.cpp` (Kacper)